LAPORAN PRAKTIKUM

POSTTEST 7

ALGORITMA PEMROGRAMAN DASAR



Disusun oleh:

Muhammad Rizky 2609106057

Kelas B2‘26

PROGRAM STUDI INFORMATIKA

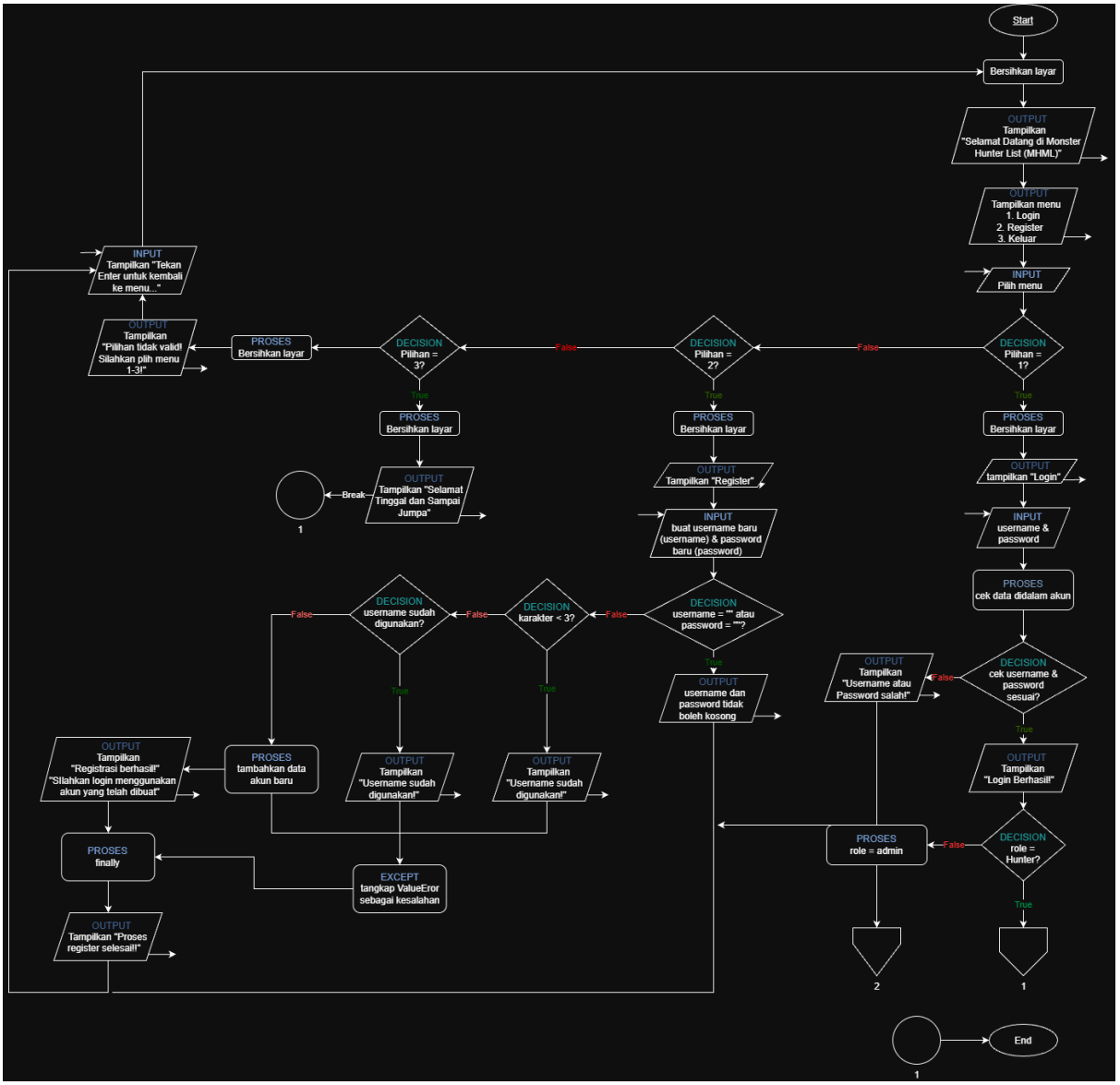
UNIVERSITAS MULAWARMAN

SAMARINDA

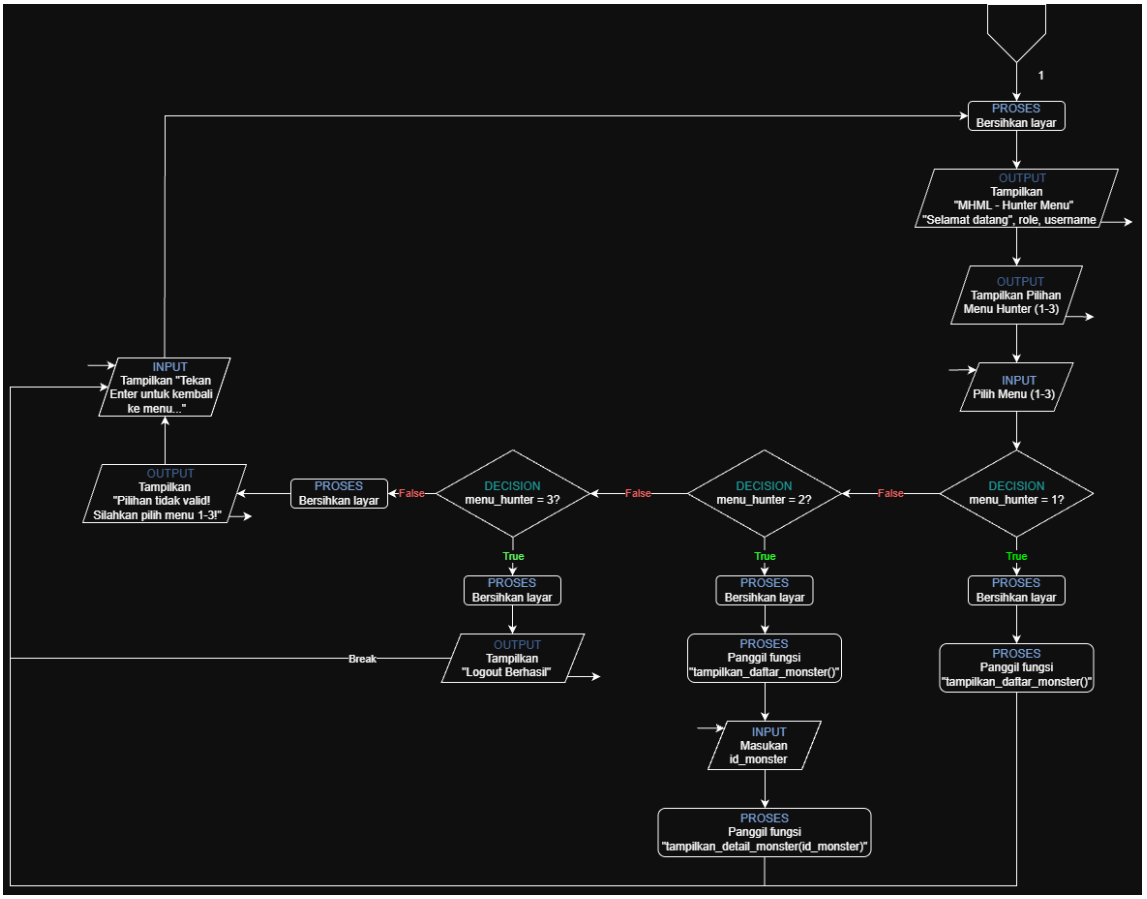
2026

1. Flowchart

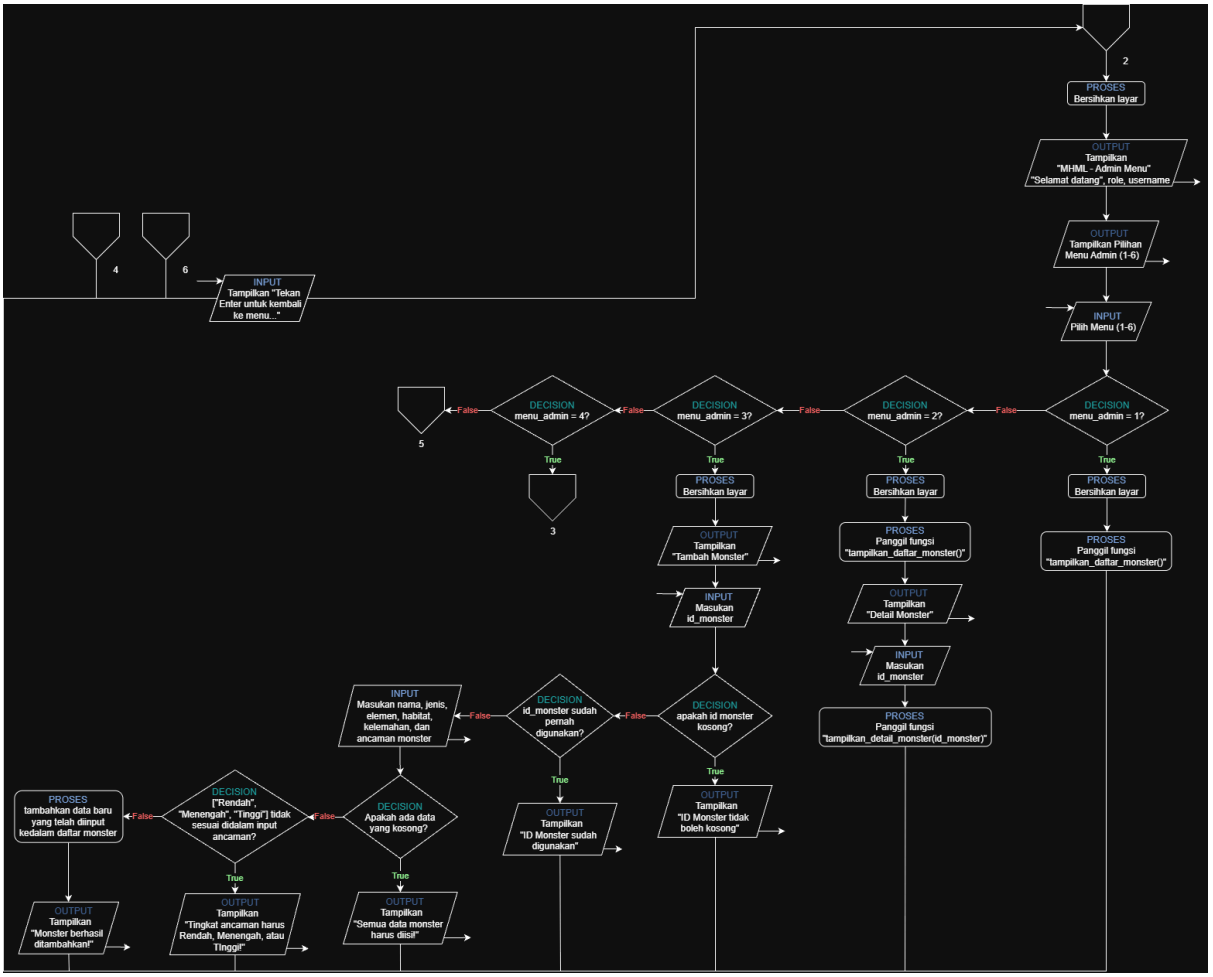
Flowchart Menu Utama



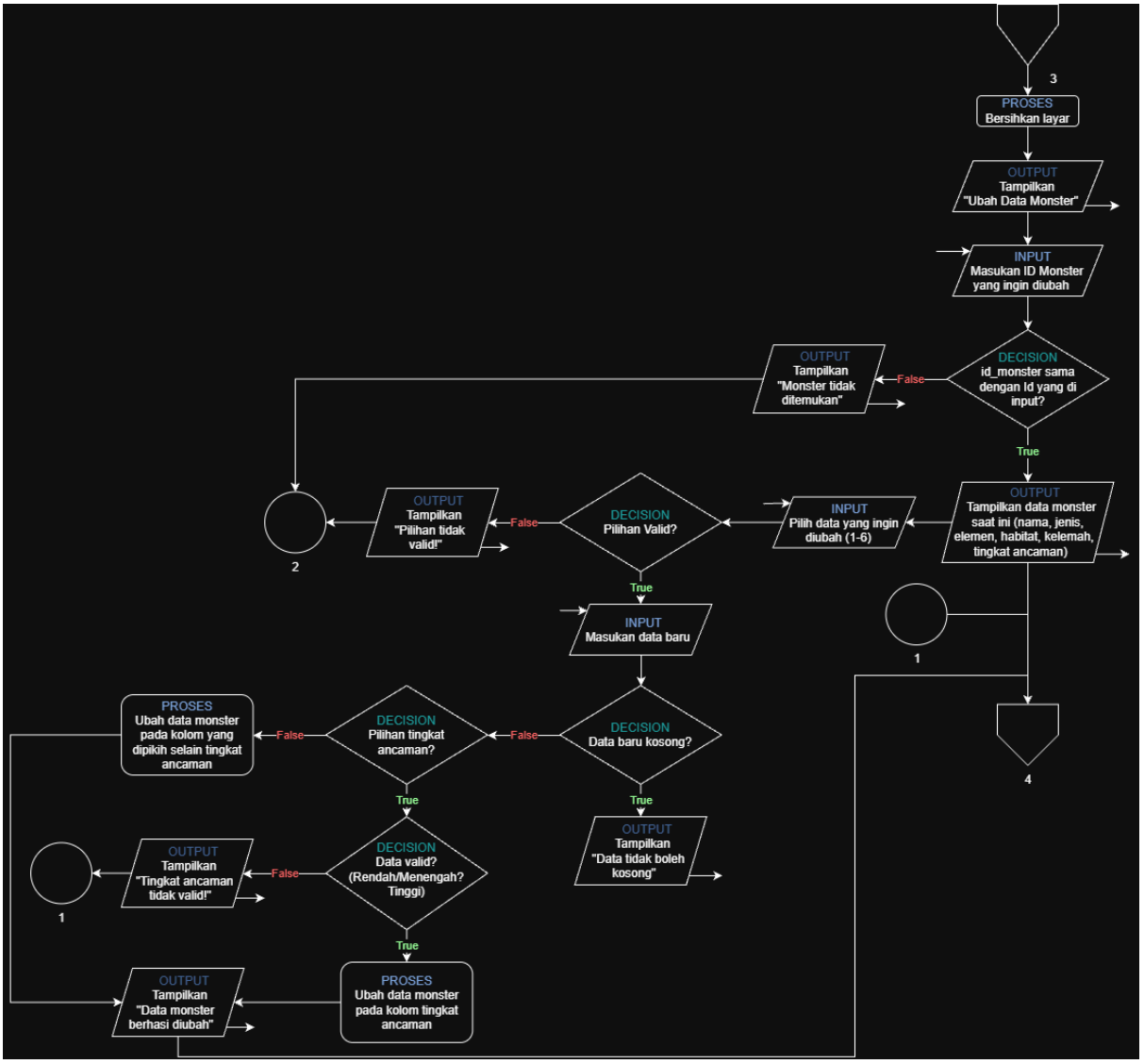
Flowchart Menu Hunter



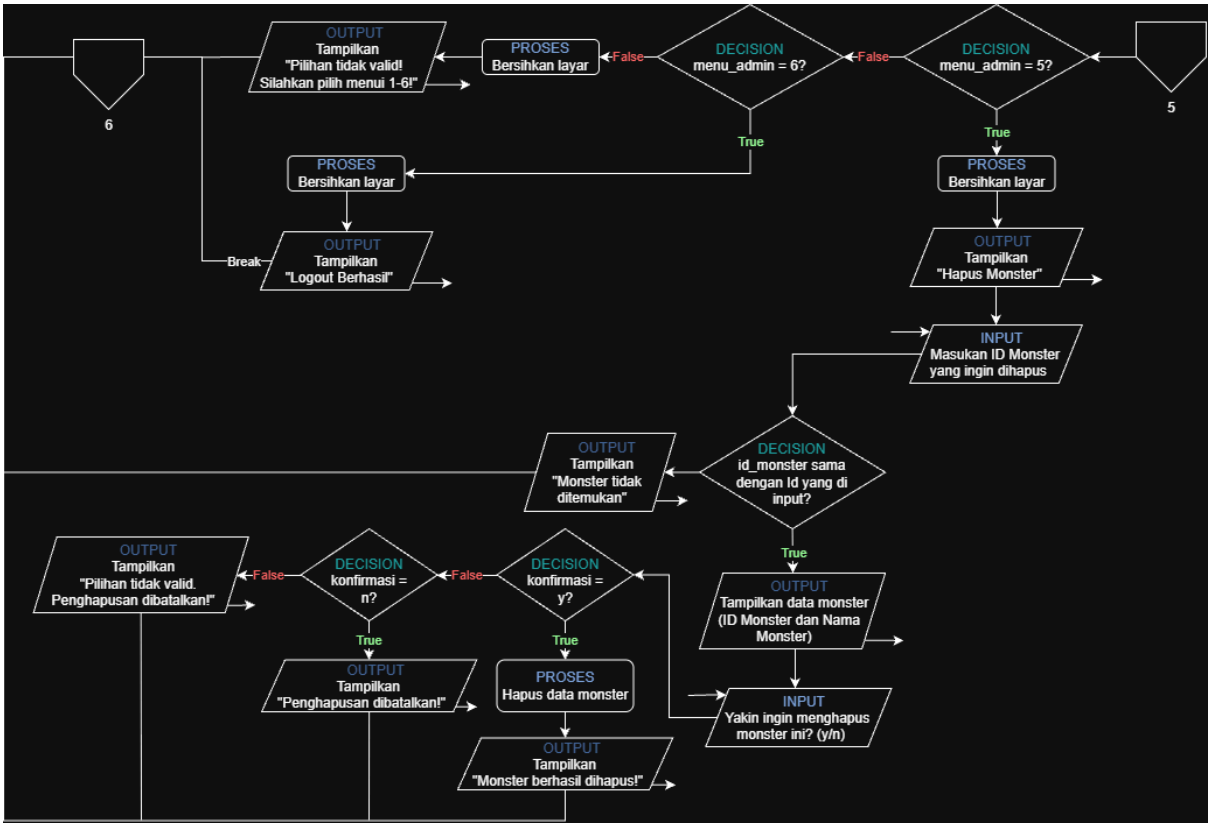
Flowchart Menu Admin



Flowchart Menu Admin Bagian Ubah Data Monster



Flowchart Menu Admin lanjutan



2. Deskripsi Singkat Program

MHML adalah program berbasis Python yang digunakan untuk mengelola dan melihat data monster. Program ini memiliki dua jenis pengguna, yaitu Admin dan Hunter. Admin dapat melihat, menambahkan, mengubah, dan menghapus data monster, sedangkan Hunter hanya dapat melihat daftar dan detail monster. Program ini juga dilengkapi fitur login, dan registrasi akun. terkhusus untuk Hunter baru, perlu melakukan registrasi akun terlebih dahulu

3. Source Code

3.1 Struktur Data Akun

```
akun = {
    "rizky": {"password": "admin123", "role": "Admin"},
    "buma": {"password": "123", "role": "Hunter"}
}
```

Dictionary akun digunakan untuk menyimpan username, password, dan role pengguna. Role menentukan hak akses pengguna sebagai Admin atau Hunter.

3.2 Struktur Data Monster

```
monster = {
    "M001": {
        "nama": "Great Izuchi",
        "jenis": "Bird Wyvern",
        "elemen": "Tidak ada",
        "habitat": "Shrine Ruins",
        "kelemahan": "Thunder, Ice",
        "ancaman": "Rendah"
    }
}
```

Dictionary monster digunakan untuk menyimpan informasi monster berdasarkan ID. Setiap monster memiliki data nama, jenis, elemen, habitat, kelemahan, dan tingkat ancaman.

3.3 Fungsi Mencari Monster

```
def cari_monster(id_monster):
    if id_monster in monster:
        return monster[id_monster]
    else:
        return None
```

Fungsi cari_monster() menerima ID monster sebagai parameter. Jika ID ditemukan, fungsi mengembalikan data monster. Jika tidak ditemukan, fungsi mengembalikan `None`.

3.4 Fungsi Validasi Tingkat Ancaman

```
def validasi_ancaman(ancaman):
    if ancaman in ["Rendah", "Menengah", "Tinggi"]:
        return True
    else:
        return False
```

Fungsi validasi_ancaman() digunakan untuk memeriksa apakah tingkat ancaman sesuai dengan pilihan yang diperbolehkan. Fungsi mengembalikan True jika valid dan False jika tidak valid.

3.5 Fungsi Menghitung Jumlah Monster

```
def hitung_monster():
    return len(monster)
```

Fungsi hitung_monster menggunakan len() untuk menghitung jumlah monster yang tersimpan dalam dictionary. Hasil perhitungan dikembalikan menggunakan return.

3.6 Fungsi Memasukkan ID Monster

```
def input_id_monster():
    id_masukan = input("Masukkan ID Monster: ").strip().upper()
    return id_masukan
```

Fungsi input_id_monster() digunakan untuk menerima ID monster dari pengguna. Metode strip() menghapus spasi di awal dan akhir input, sedangkan upper() mengubah huruf menjadi kapital.

3.7 Prosedur Menampilkan Daftar Monster

```
def tampilkan_daftar_monster():
    print("================ DAFTAR MONSTER ================")
    daftar_id = list(monster.keys())
    tampilkan_monster_rekursif(daftar_id)
    print("Jumlah monster:", hitung_monster())
```

Prosedur tampilkan_daftar_monster() digunakan untuk menampilkan daftar monster. ID monster diambil menggunakan keys(), kemudian diubah menjadi list. Daftar ditampilkan menggunakan fungsi rekursif, sedangkan jumlah monster diperoleh dari fungsi hitung_monster().

3.8 Prosedur Menampilkan Detail Monster

```
def tampilkan_detail_monster(id_monster):
    data_monster = cari_monster(id_monster)

    if data_monster is not None:
        print("ID Monster:", id_monster)
        print("Nama Monster:", data_monster["nama"])
    else:
        print("Monster tidak ditemukan!")
```

Prosedur tampilkan_detail_ monster() menerima ID monster sebagai parameter dan menggunakan fungsi cari_monster() untuk mendapatkan datanya. Jika ditemukan, informasi monster ditampilkan. Jika tidak, program menampilkan pesan bahwa monster tidak ditemukan.

3.9 Fungsi Rekursif

```
def tampilkan_monster_rekursif(daftar_id, indeks=0):
    if indeks >= len(daftar_id):
        return

    id_data = daftar_id[indeks]
    print(id_data, "-", monster[id_data]["nama"])

    tampilkan_monster_rekursif(daftar_id, indeks + 1)
```

Fungsi tampilkan_monster_rekursif() menampilkan daftar monster dengan memanggil dirinya sendiri. Variabel indeks digunakan untuk menentukan posisi monster yang ditampilkan. Rekursi berhenti ketika indeks mencapai jumlah data monster.

3.10 Fitur Error Handling

```
try:
    if username == "" or password == "":
        raise ValueError("Username dan password tidak boleh
kosong!")
    elif len(username) < 3:
        raise ValueError("Username minimal 3 karakter!")
    elif username in akun:
        raise ValueError("Username sudah digunakan!")

except ValueError as kesalahan:
    print("Registrasi gagal:", kesalahan)
```

```
else:
    akun[username] = {
        "password": password,
        "role": "Hunter"
    }
    print("Registrasi berhasil!")

finally:
    print("Proses register selesai.")
```

Error Handling digunakan untuk menangani kesalahan saat registrasi. try menjalankan pemeriksaan input, sedangkan raise menghasilkan kesalahan jika syarat tidak terpenuhi. except menangkap kesalahan, else menjalankan registrasi jika tidak ada kesalahan, dan finally dijalankan terlepas dari berhasil atau gagalnya proses registrasi.

3.11 Fitur Login

```
if username in akun and akun[username]["password"] == password:
    pengguna_aktif = username
    role_aktif = akun[username]["role"]
```

Kode ini memeriksa username dan password pengguna. Jika sesuai, username dan role disimpan ke variabel global yang digunakan untuk mengenali pengguna aktif.

3.12 Fitur Register

```
akun[username] = {
    "password": password,
    "role": "Hunter"
}
```

Kode ini menyimpan akun baru ke dalam dictionary `akun` *dengan role Hunter. Sebelum akun disimpan, program memeriksa input kosong, panjang username, dan kemungkinan username sudah digunakan.*

3.13 Fitur Menambahkan Monster (Create)

```
monster[id_monster] = {
    "nama": nama,
    "jenis": jenis,
    "elemen": elemen,
    "habitat": habitat,
```

```
    "kelemahan": kelemahan,
    "ancaman": ancaman
}
```

Fitur Create digunakan untuk menambahkan monster baru ke dictionary monster. Sebelum data disimpan, program memeriksa ID, kelengkapan data, dan validitas tingkat ancaman.

3.14 Fitur Mengubah Data Monster (Update)

```
kolom = {
    "1": "nama",
    "2": "jenis",
    "3": "elemen",
    "4": "habitat",
    "5": "kelemahan",
    "6": "ancaman"
}

kunci = kolom[pilihan_ubah]
data[kunci] = data_baru
```

Fitur Update digunakan untuk mengubah informasi monster. Dictionary kolom menghubungkan pilihan menu dengan atribut monster, sedangkan data[kunci] = data_baru menyimpan nilai baru pada atribut yang dipilih.

3.15 Fitur Menghapus Monster (Delete)

```
if konfirmasi == "y":
    del monster[id_monster]
    print("Monster berhasil dihapus!")
```

Fitur Delete digunakan untuk menghapus data monster berdasarkan ID yang dipilih Admin. Perintah del menghapus data setelah Admin memberikan konfirmasi.

3.16 Variabel Global dan Lokal

```
pengguna_aktif = ""
role_aktif = ""
jumlah_login_gagal = 0
```

Variabel tersebut dideklarasikan di luar fungsi sehingga termasuk variabel global. Program juga menggunakan variabel global akun dan monster untuk menyimpan data akun serta monster.

Contoh variabel lokal:

```
def cari_monster(id_monster):
    if id_monster in monster:
        return monster[id_monster]
    else:
        return None
```

Parameter id_monster digunakan di dalam fungsi cari_monster(). Contoh variabel lokal lainnya adalah data_monster, daftar_id, indeks, dan id_data yang digunakan dalam fungsi atau prosedur masing-masing.

3.17 Fitur Multiuser Admin dan Hunter

```
if role == "Hunter":
    # Menu Hunter
    pass

elif role == "Admin":
    # Menu Admin
    pass
```

Fitur multiuser membedakan hak akses pengguna berdasarkan role. Hunter hanya dapat melihat daftar dan detail monster, sedangkan Admin dapat melihat, menambahkan, mengubah, dan menghapus data monster.

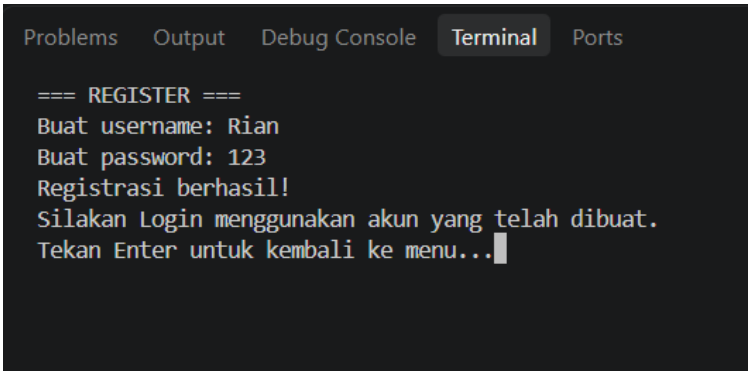
3.18 Fitur Logout

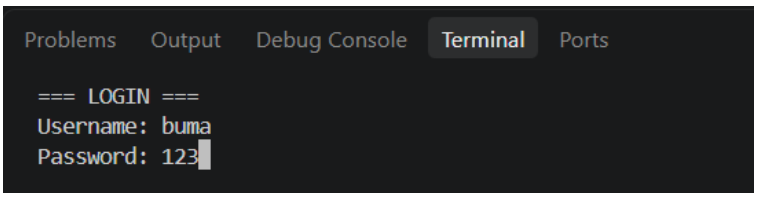
```
print("Logout berhasil!")
break
```

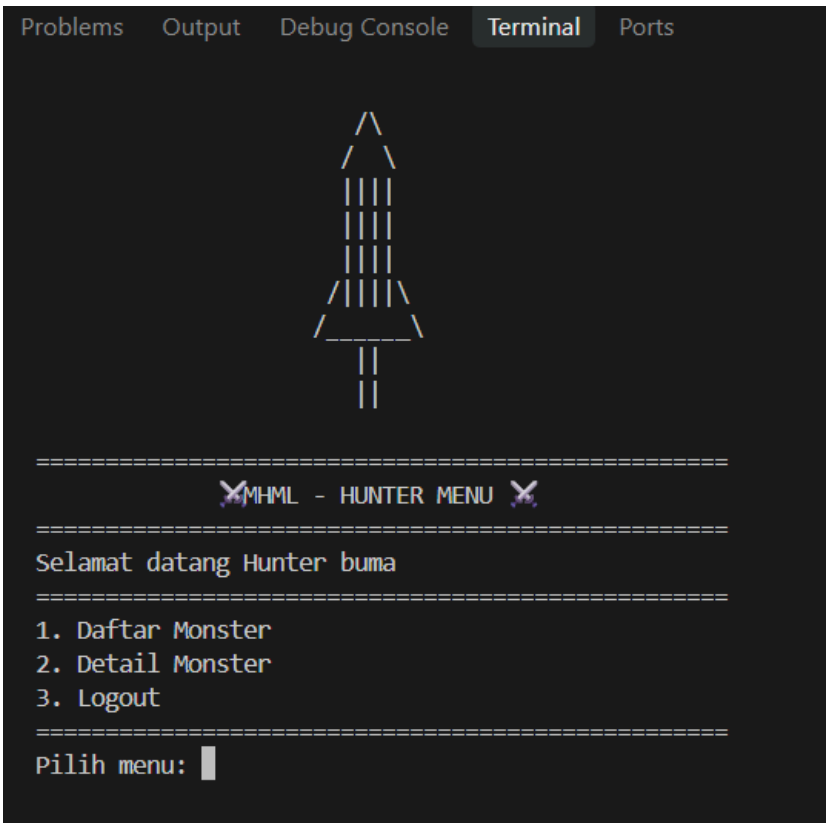
Fitur Logout digunakan untuk keluar dari menu Admin atau Hunter. Perintah break menghentikan perulangan menu sehingga pengguna kembali ke menu utama program.

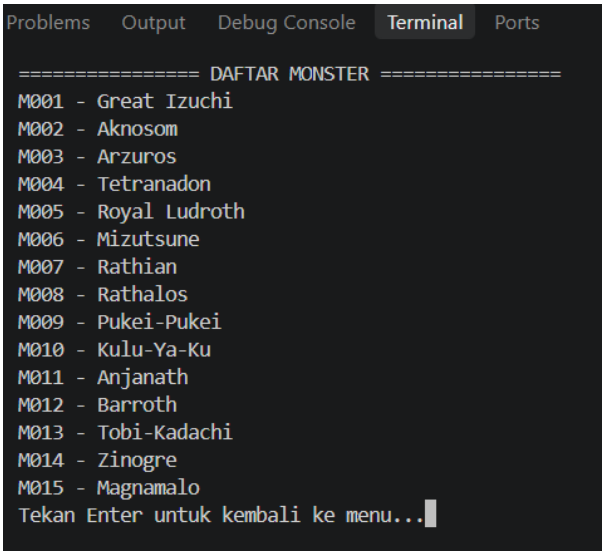
4. Hasil Output

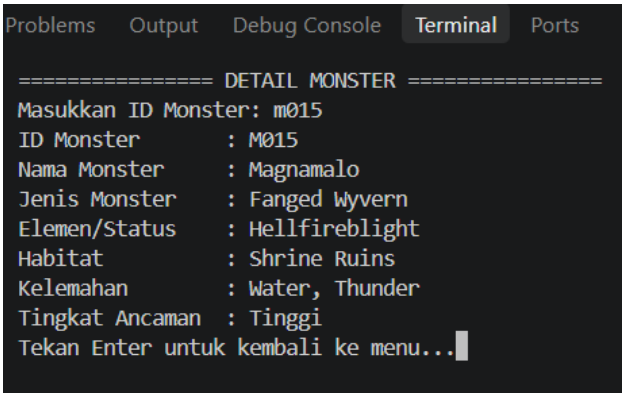


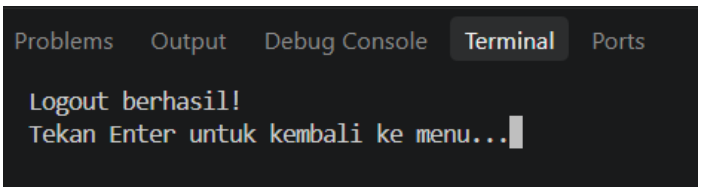


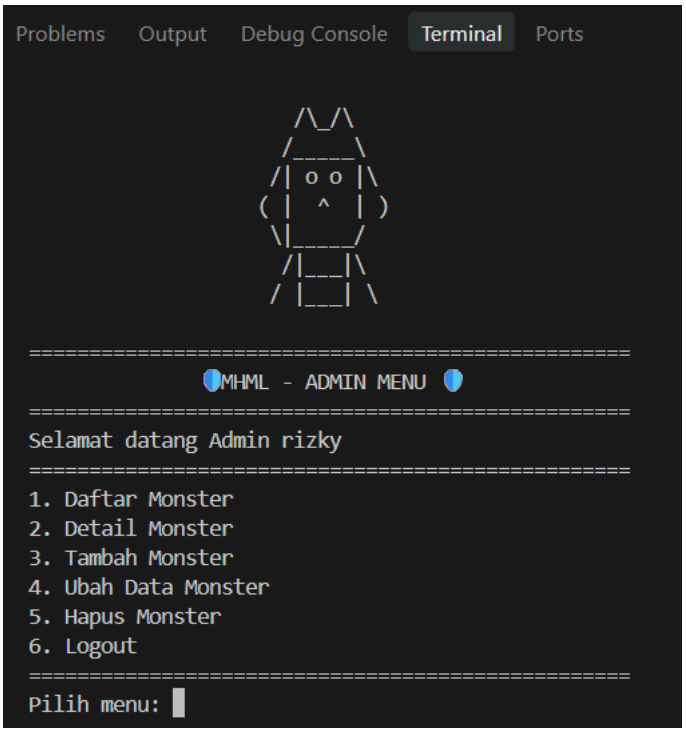


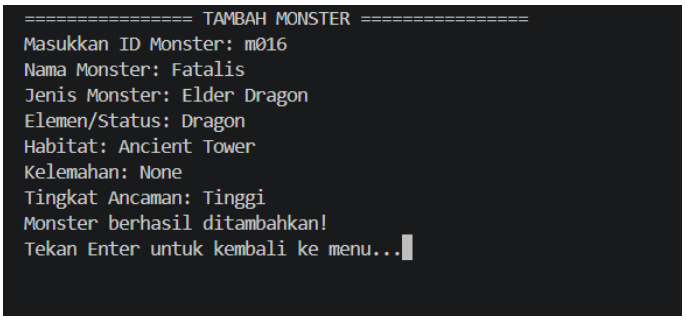

Problems
Output
Debug Console
Terminal
Ports
/\
/ \
/||||\
/_____\
||
||
==========
⚔MHML - HUNTER MENU ⚔
==========
Selamat datang Hunter buma
==========
1. Daftar Monster
2. Detail Monster
3. Logout
==========
Pilih menu:

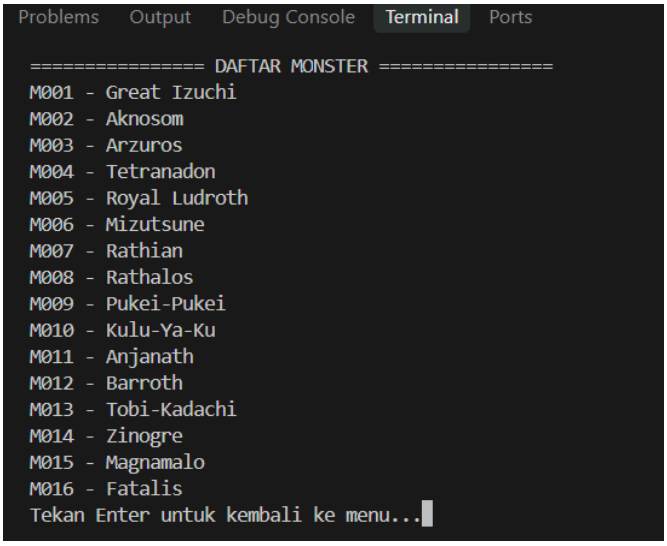

Problems
Output
Debug Console
Terminal
Ports
========== DAFTAR MONSTER ==========
M001 - Great Izuchi
M002 - Aknosom
M003 - Arzuros
M004 - Tetranadon
M005 - Royal Ludroth
M006 - Mizutsune
M007 - Rathian
M008 - Rathalos
M009 - Pukei-Pukei
M010 - Kulu-Ya-Ku
M011 - Anjanath
M012 - Barroth
M013 - Tobi-Kadachi
M014 - Zinogre
M015 - Magnamalo
Tekan Enter untuk kembali ke menu...

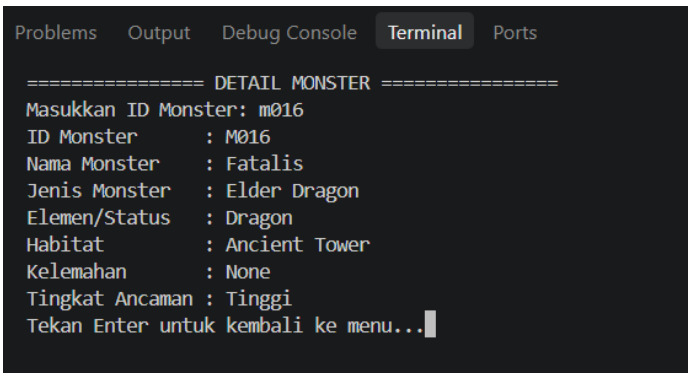

Problems
Output
Debug Console
Terminal
Ports
========== DETAIL MONSTER ==========
Masukkan ID Monster: m015
ID Monster : M015
Nama Monster : Magnamalo
Jenis Monster : Fanged Wyvern
Elemen/Status : Hellfireblight
Habitat : Shrine Ruins
Kelemahan : Water, Thunder
Tingkat Ancaman : Tinggi
Tekan Enter untuk kembali ke menu...

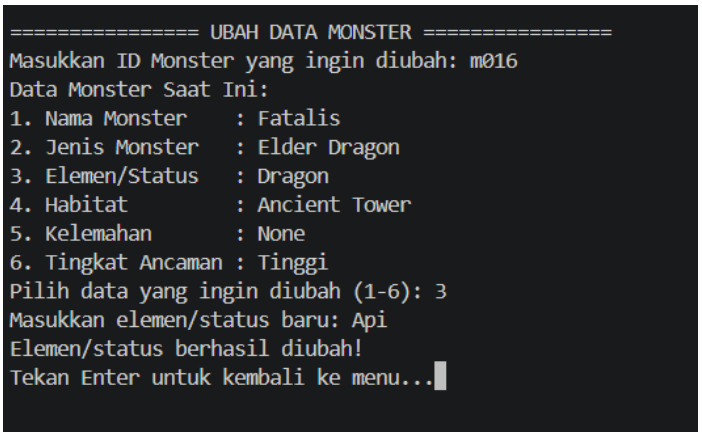

Problems
Output
Debug Console
Terminal
Ports
Logout berhasil!
Tekan Enter untuk kembali ke menu...

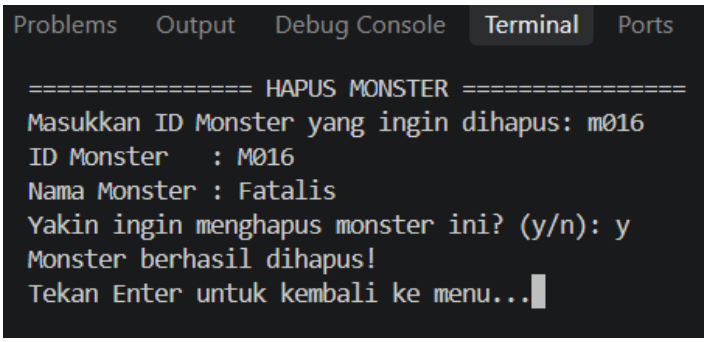

Problems
Output
Debug Console
Terminal
Ports
/_/\
/_____\
/| o o |\
(| ^ |)
\|_____/
/|___|\
/ |___| \
MHML - ADMIN MENU
Selamat datang Admin rizky
1. Daftar Monster
2. Detail Monster
3. Tambah Monster
4. Ubah Data Monster
5. Hapus Monster
6. Logout
Pilih menu:


TAMBAH MONSTER
Masukkan ID Monster: m016
Nama Monster: Fatalis
Jenis Monster: Elder Dragon
Elemen/Status: Dragon
Habitat: Ancient Tower
Kelemahan: None
Tingkat Ancaman: Tinggi
Monster berhasil ditambahkan!
Tekan Enter untuk kembali ke menu...

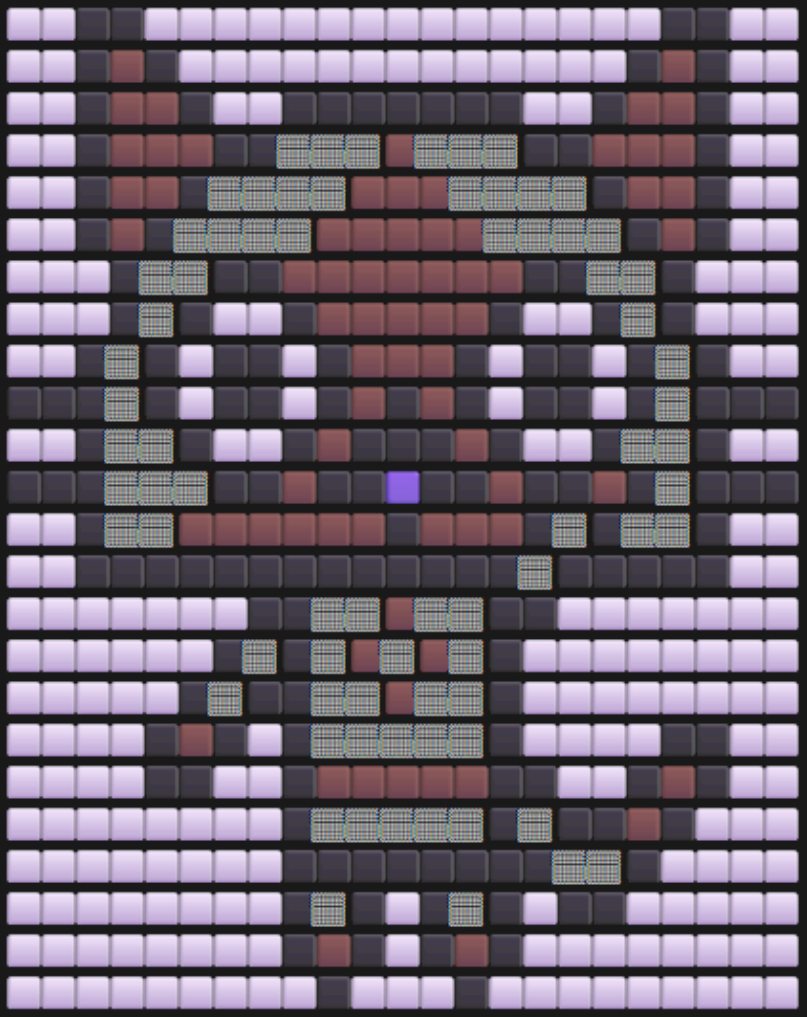

Problems
Output
Debug Console
Terminal
Ports
DAFTAR MONSTER
M001 - Great Izuchi
M002 - Aknosom
M003 - Arzuros
M004 - Tetranadon
M005 - Royal Ludroth
M006 - Mizutsune
M007 - Rathian
M008 - Rathalos
M009 - Pukei-Pukei
M010 - Kulu-Ya-Ku
M011 - Anjanath
M012 - Barroth
M013 - Tobi-Kadachi
M014 - Zinogre
M015 - Magnamalo
M016 - Fatalis
Tekan Enter untuk kembali ke menu...


Problems
Output
Debug Console
Terminal
Ports
========== DETAIL MONSTER ==========
Masukkan ID Monster: m016
ID Monster : M016
Nama Monster : Fatalis
Jenis Monster : Elder Dragon
Elemen/Status : Dragon
Habitat : Ancient Tower
Kelemahan : None
Tingkat Ancaman : Tinggi
Tekan Enter untuk kembali ke menu...


========== UBAH DATA MONSTER ==========
Masukkan ID Monster yang ingin diubah: m016
Data Monster Saat Ini:
1. Nama Monster : Fatalis
2. Jenis Monster : Elder Dragon
3. Elemen/Status : Dragon
4. Habitat : Ancient Tower
5. Kelemahan : None
6. Tingkat Ancaman : Tinggi
Pilih data yang ingin diubah (1-6): 3
Masukkan elemen/status baru: Api
Elemen/status berhasil diubah!
Tekan Enter untuk kembali ke menu...


Problems
Output
Debug Console
Terminal
Ports
========== HAPUS MONSTER ==========
Masukkan ID Monster yang ingin dihapus: m016
ID Monster : M016
Nama Monster : Fatalis
Yakin ingin menghapus monster ini? (y/n): y
Monster berhasil dihapus!
Tekan Enter untuk kembali ke menu...

Problems Output Debug Console **Terminal** Ports



```
Selamat Tinggal dan Sampai Jumpa
PS C:\Users\USER\Documents\Praktikum\Praktikum_APD_B2_2026>
```

5. Langkah-langkah Git

5. 1 Git Add

```
PS C:\Users\USER\Documents\Praktikum\Praktikum_APD_B2_2026> git add .
```

Memindahkan perubahan file dari direktori kerja (Working Directory) ke area penampungan sementara yang disebut stagging area

5.2 Git Commit

```
PS C:\Users\USER\Documents\Praktikum\Praktikum_APD_B2_2026> git commit -m "Update Posttest 5"
[main a86a953] Update Posttest 5
 4 files changed, 680 insertions(+)
 rename Kelas/{Pertemuan_Ke4 => Pertemuan ke4}/main.py (100%)
 create mode 100644 Kelas/Pertemuan ke5/main.py
 create mode 100644 Kelas/Pertemuan ke6/main.py
 create mode 100644 POSTTEST/POSTTEST_5/2609106057-Muhammad_Rizky-PT-5.py
```

Tindakan menyimpan snapshot dari proyek kita. Setiap kali kita membuat commit, kita menyimpan riwayat perubahan yang telah kita lakukan.

5.3 Git Push

```
PS C:\Users\USER\Documents\Praktikum\Praktikum_APD_B2_2026> git push -u origin main
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 16 threads
Compressing objects: 100% (8/8), done.
Writing objects: 100% (10/10), 5.69 KiB | 1.89 MiB/s, done.
Total 10 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote: This repository moved. Please use the new location:
remote:   https://github.com/MoreWayPixel/Praktikum_APD_B2_2026.git
To https://github.com/MoreWayPixel/MoreWayPixel-Praktikum_APD_B2_2026.git
   09fa041..a86a953  main -> main
branch 'main' set up to track 'origin/main'.
```

Setelah menyimpan perubahan secara lokal, kita bisa mengunggahnya ke Github menggunakan ‘push’ agar bisa diakses dari mana saja.